

1. Show us a cool thing you've built/coded up :) I know you're already submitted a portfolio, but this one should be something that you're especially proud of and want to explain why. For reference a single image and a three sentence description explaining why it's cool / unusual is perfect. If it is a school project explain what class it was for

For my PhD research, I built a bare-metal, ultra-low-power telemetry system in C that simultaneously processed noisy Bio signals (EDA-ECG-EMG) on the edge. To compensate for strict battery constraints, I designed an interrupt-driven architecture that bypassed floating-point math entirely, using bit-shifted IIR filters to clean the chaotic analog data. Instead of continuously transmitting raw data, the firmware actively triages the signal, waking the radio to transmit only when a significant biological event occurs, saving massive amounts of power and bandwidth.

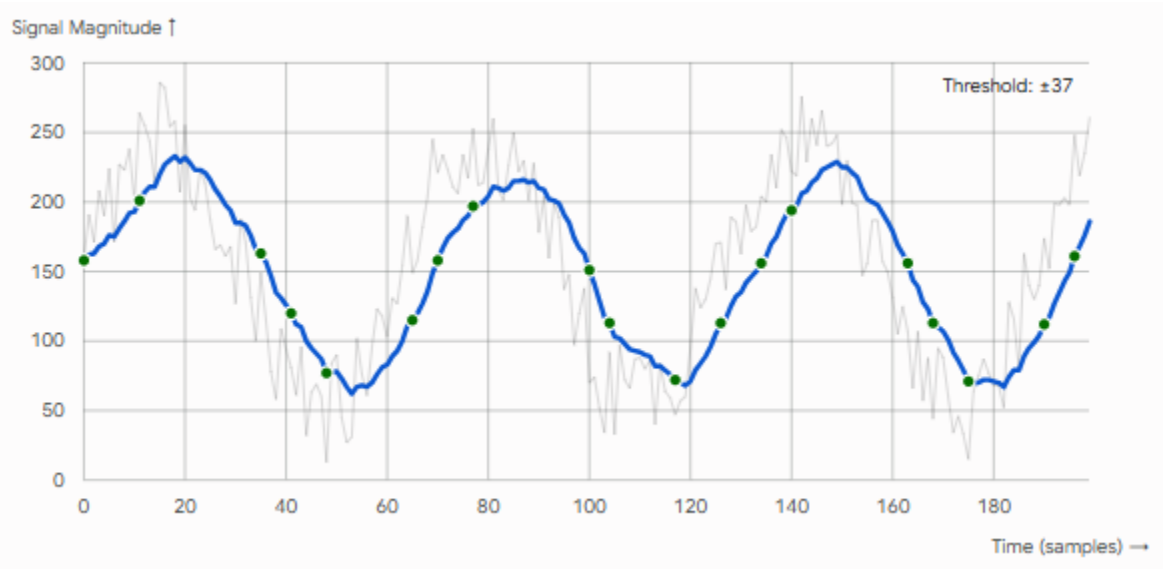
The code uses bitwise shifting ($\gg 3$) instead of division.

```
// 1. Simple IIR Filter with bitwise math.
// We do not use floating-point numbers so we can save CPU and battery life.
// It is the same as:  $y[n] = y[n-1] + (x[n] - y[n-1]) / 8$ 
filtered_signal += (raw_data - filtered_signal) >> 3;

// 2. Delta Compression trick.
// Only turn on the radio if the data changed a lot, because the radio uses too much power.
int32_t delta = filtered_signal - last_transmitted_signal;

if (abs(delta) > NOISE_THRESHOLD) {
    wake_radio_from_sleep();
    transmit_telemetry(filtered_signal);
    power_down_radio();

    last_transmitted_signal = filtered_signal;
}
```



2. Describe your setup. What text editor / IDE / cosmic ray do you use? Why do you like it? How has it changed over time?

STM32CubeIDE is my go-to for setting up the clock trees, handling pinouts and hardware debugging, but I usually write the actual C logic in a lightweight editor like VS Code.

I use this combination because it gives me the best of both worlds. STM32CubeIDE is very powerful for embedded hardware, but like many heavy IDEs, it can feel slow and clunky just for typing text. VS Code is incredibly lightweight, opens instantly, and has great syntax highlighting. Opening VS Code just feels much more comfortable and less frustrating for long coding sessions.

When I first started, I tried to do absolutely everything inside the heavy, hardware-specific IDEs. I thought I had to use them for every step. But over time, as my projects grew, I realized that writing thousands of lines of code in a slow editor is exhausting. I have stayed with the hardware IDEs because you need them for deep debugging and chip configuration, but moving my pure text editing to a light tool like VS Code completely improved my workflow and speed.

3. You get a PCB that resets randomly once every few hours. What are the steps you take to debug it? What are your go-to first couple of tests? When do you change strategies? Bonus points if you can tell a story about a real incident

I grab an Arduino Uno and connect its GND to the target PCB's GND, then connect the PCB's RESET line to Arduino Pin 2. I use Pin 2 because it supports hardware interrupts, so the Arduino reacts instantly when the reset line drops instead of constantly polling.

Then I load a simple sketch that attaches a FALLING interrupt to Pin 2. Every time the target board resets, the Arduino records the time since the previous reset using `millis()` and prints it to the serial port. I leave the Arduino connected to my PC and log the serial output to a text file for a day or two.

When I check the log, the pattern tells me a lot. If the resets happen at random intervals, it's likely a hardware issue, noise, power instability, temperature or a bad connection. But if the resets occur at nearly the exact same interval every time, that's a strong sign of a software issue, such as a watchdog timeout, memory leak or counter overflow.

All I did was log reset timing but it quickly tells me whether I should be looking at hardware or software.

4. What do you think firmware/embedded engineers do wrong? What are common words of wisdom/best practices you disagree with? Why?

Real-world electrical signals are rarely perfect digital steps; they usually contain noise and rapid fluctuations. When you use software loops to monitor these signals, like repeatedly checking a pin in your main code, you create a limited sampling rate. This acts like a blinking camera shutter: if a very fast hardware glitch occurs between those checks, the software misses it entirely. To build a reliable system, you should avoid software polling for critical events and use hardware interrupts instead. Interrupts are dedicated circuits in the microcontroller that instantly detect pin changes the moment they happen, ensuring you capture fast events without relying on the timing of your software loop.

5. Have you ever hacked together a firmware "fix" you knew was kinda sketchy but it kept the project alive? What was it? Bonus points if it's one you're secretly proud of.

you might hate this answer, but I'm just going to shoot straight and give you my best one.

We had a sudden deadline to present a working board to a community meeting and my phd advisor needed to see live data streaming. The proper architecture called for a highly optimized binary payload, bit-packing the sensor data into strict data structs to maximize the Bluetooth throughput.

But I didn't have the days it would take to write and debug the serialization on the firmware side and build a custom parser on the receiver side. So, my sketchy fix? I ditched the binary protocol entirely and just used `printf` to cast all the raw data into ASCII strings.

Instead of sending an efficient hex value bit-by-bit, I was transmitting the literal text characters of the numbers. The bandwidth overhead was absolutely atrocious and it was a brutally inefficient use of the BLE stack. But because it was just standard text, the receiver could read it natively without a custom parser.

It was a massive hack, but it ran flawlessly for the exact two-minute window we needed to secure the room's admiration. But honestly? I wasn't secretly proud of it; I was terrified. I kept thinking, 'What if they want me to demo it again on Monday?' I couldn't leave that ticking time bomb in the firmware. So, I spent my entire Saturday and Sunday locked in the lab, ripping out the ASCII hack and properly writing the binary parser. I survived the Friday deadline but I spent the weekend making sure I did it right. Hmmm. I think I'm proud of it now because I had a fun story to tell you guys!